CrossTwin — System Overview

Django + PostGIS digital twin for Dutch urban data (EPSG:28992), with a separate TiTiler process for raster tiles

XYZ raster tiles (direct)

BROWSER

Map UI
Mapbox GL JS
Layers.js · mainMap.js
LstTool.js · SvfTool.js

Indicator panels
HTMX swaps (hx-get)
water · heat · housing · builtup
what-if sliders + switches

Map editing
Editing.js +
Mapbox GL Draw
(staff only)

Population dock
forecast curve +
what-if controls

Django admin
/admin/ (staff login)

JSON / HTML

HTMX

DJANGO · DIGITALTWIN :8000

URL router (DigitalTwin/urls.py)
/api/ · /watersupply/ · /housing/ · /urban_heat/ · /physicalEnv/ · /weather/ · /importer/

mainMap
layer catalog /api/layers/
GeoJSON + bounds endpoints
admin-unit & population panels
map editing (core/editing.py)

Domain apps
watersupply · housing · urban_heat
administrative · physicalEnv
builtup · nature · weather · Energy
on demand: LST (lst/) · SVF

core
MODEL / VECTOR / RASTER /
WMS registries (utils.py)
rasterOperations → COG
signals · cache versioning

importer
file upload (GeoJSON/SHP)
external catalog (70 sets)
SpatialParentIndex
BulkWriter · deferred_cascades

weather WMS proxy
/weather/wms/<name>/tile/
keep-alive session
24 h cache for TIME= frames

Caches (LocMem)
geojson — 2 min, ≤ 8 MB
wms_tiles — 24 h / 5 min
default

Signals keep derived data consistent: population & urban-area cascade, watersupply indicator chain, raster → COG export, layer cache bumps.

/cog/info

TiTiler :8001
FastAPI · tiler.py
/cog/tiles/{z}/{x}/{y}
/cog/info

read COG

COG store
cogs/<app_label>/*.tif
EPSG:4326, rio-cogeo
written by post_save

fetch datasets

WMS GetMap

ORM + raw SQL

PostgreSQL 16 + PostGIS
84 models · GeometryField & RasterField (srid 28992) · GiST indexes · pg_stat_statements

EXTERNAL

- PDOK
- CBS
- BIS Nederland
- OpenStreetMap
- Sentinel-2
- Earth Engine
- KNMI
- RIVM
- Klimaateffectatlas
- WMS servers

Browser · Django app · Raster / tiles · Cache · Storage · External source

Django Apps & the Model Registry

CrossTwin · architecture

core/utils.py::build_model_registry() scans nine domain apps; every generic endpoint is driven by the registries

Django · Vector · Raster · WMS

DOMAIN APPS (ALLOWED_APPS) — MODELS · CALCULATIONS.PY · VIEWS

administrative (5)
Province › City › District › Neighborhood
PopulationProjection

physicalEnv (17)
LandCover · DEM · DSM
soil map · groundwater
HILUCS · soil.py (SCS)
svf.py · UrbanFabricCut

watersupply (17)
extraction · treatment
pipes · NRW · OPEX
SupplySecurity

housing (9)
supply/demand · HPI
mortgage · rentals
affordability
social profile (KEA)

urban_heat (12)
UTCI · PET · Tmrt
LST · SVF · SUHII
NBS · WBGT · lst/ SMW
heat per buurt (KEA)

builtup (7)
ZoningArea (HILUCS)
Street · Park · Facility
Building · Property
climate risk (KEA)

nature (7)
green / blue
layers · trees

weather (7)
WMS layers
time steps

Energy (3)
energy labels

Number tags = model classes per app. Indicator dashboards (watersupply, housing, urban_heat, builtup) follow calculations.py → views._build_indicators() → HTMX partial; MOCK_DATA fills in when a province is missing.

`DAG edges documented in calculations.py docstrings → core/DAG.dot (DPSIR)`

scan models

build_model_registry()
core/utils.py — scans allowed_apps at startup

has GeometryField, no RasterField → VECTOR
has RasterField → RASTER · 'WMS' in key → WMS

MODEL_REGISTRY
all models

VECTOR_REGISTRY
GeoJSON, bounds, editing

RASTER_REGISTRY
COG export, tile URLs

WMS_REGISTRY
WMS layer catalog

importer
field mapping, upsert keys

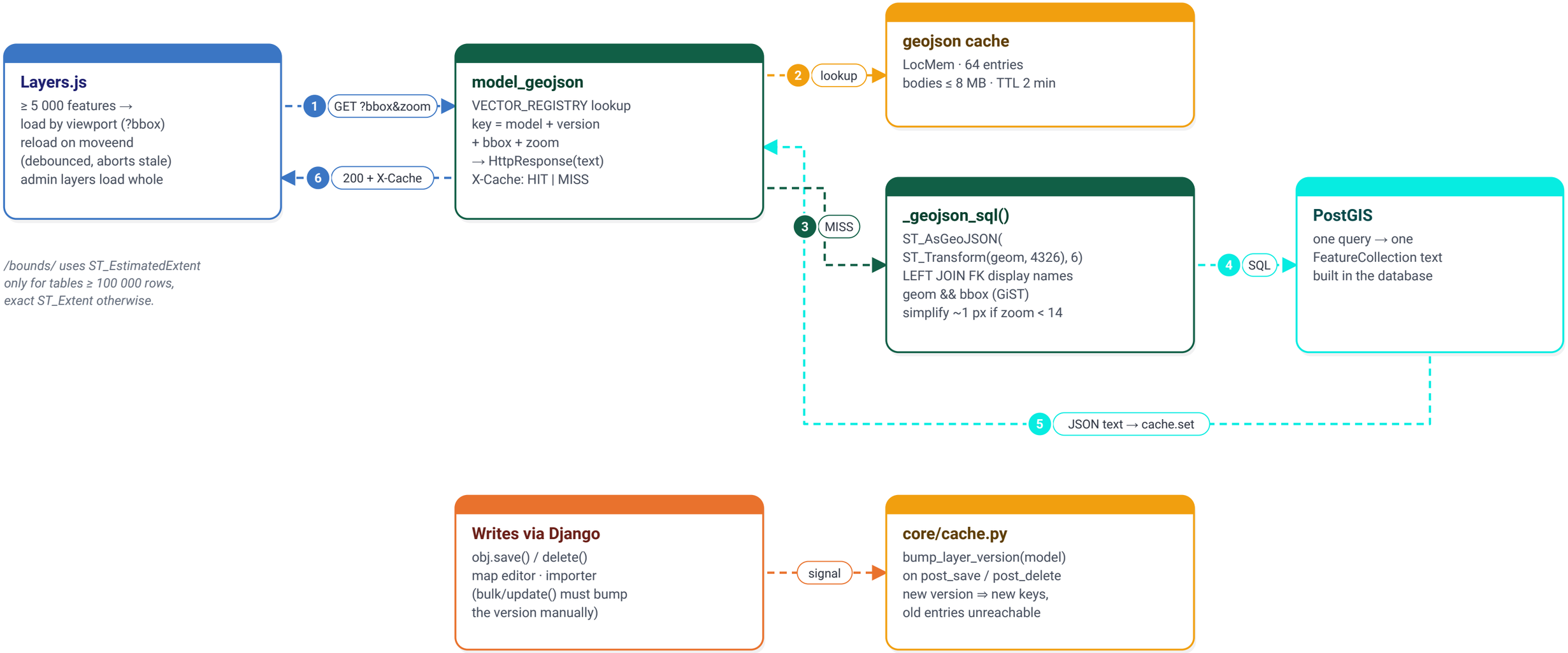
mainMap
/api/layers/, GeoJSON, editing

core.signals
post_save → COG, cache bump

weather
WMS catalog, tile proxy

Vector Layer Request — GeoJSON Path

mainMap.views.model_geojson: one SQL statement per request, cached by per-model version

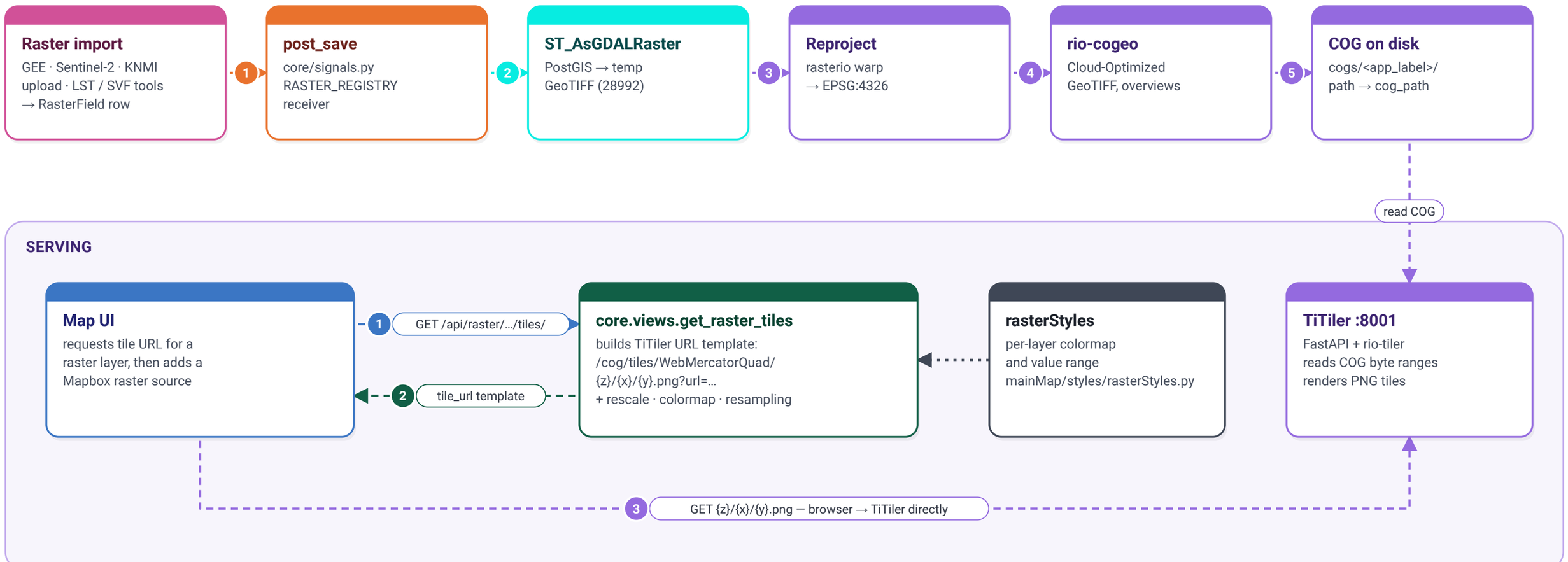


/bounds/ uses ST_EstimatedExtent only for tables ≥ 100 000 rows, exact ST_Extent otherwise.

Edits from QGIS/psql bypass Django signals → visible only after the 2-min TTL expires.
LocMem is per process: with several workers, point CACHES at Redis so a bump reaches all of them.

Raster Pipeline — PostGIS Raster → COG → Tiles

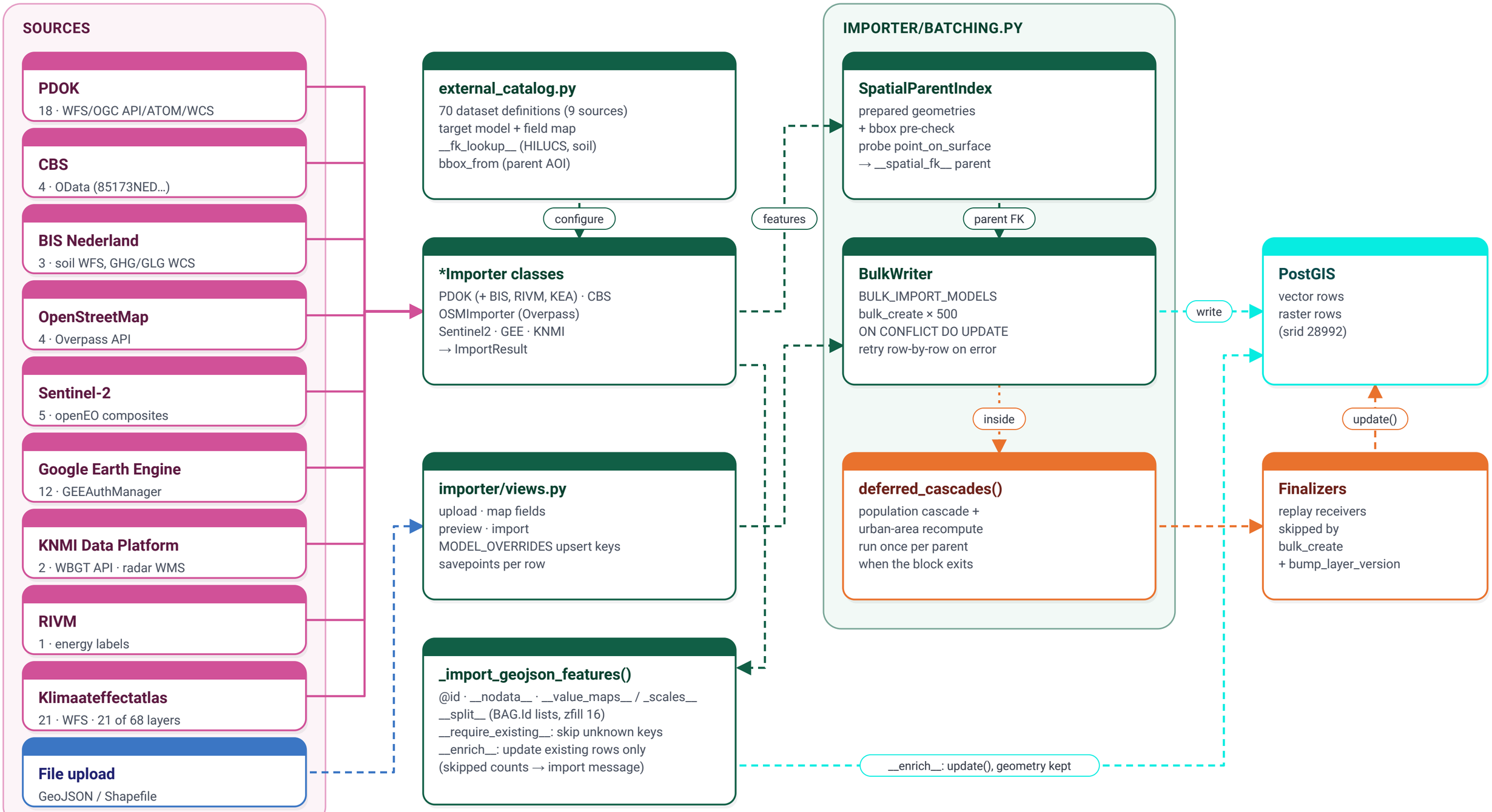
core/rasterOperations.py::export_raster_to_cog() runs from a post_save signal on every RASTER_REGISTRY model



Heavy libs (rasterio, rio-cogeo, numpy) are imported inside export_raster_to_cog(), never at module level, so Django startup and the URLconf stay fast; tests patch core.rasterOperations.export_raster_to_cog.

Data Import Pipeline

Two entry points — file upload and the catalog-driven external importers — share batching and deferred cascades



A model may be in BULK_IMPORT_MODELS only if it has no save() override (BulkImportRegistryTests) —
e.g. LandCoverVector, Street, Facility, SoilArea, the nature layers, Tree; a new post_save receiver on one needs its finalizer too.
Known gap: _generic_import (file upload) maps geometry/raster fields only — attribute columns are not imported.
__enrich__ layers (KEA per building) write attributes onto existing rows with update(), never the geometry, and bump the cache of every registry model on that table.

External source · User upload · Importer code · Cascades · Storage

Signal-Driven Derived Data

post_save / post_delete receivers keep aggregates and indicator chains consistent; update() avoids recursion

ADMINISTRATIVE/SIGNALS.PY

Neighborhood
saved / deleted

_recompute_population()

District
currentPopulation · density · urban_area

update()

City
currentPopulation · density · urban_area

update()

Province
currentPopulation · density · urban_area

Sums both population and urban_area per level.

re-enter cascade

PHYSICALENV/SIGNALS.PY

Building saved / deleted (editor, id < 0)
building_created / _deleted · New_Units → LandCover

urbanize_around() / release_ring()
new: 10 m buffer cut from latest land cover
moved / deleted: UrbanFabricCut pieces returned
landcover_rewriting → mainMap journal (Restore)

LandCoverVector saved
schedule_urban_area_recompute(city_id)

One set-based UPDATE … RETURNING
ST_Intersection with urban-fabric polygons
(latest year), skipped when fully inside;
ST_Area in storage-CRS metres

CORE/SIGNALS.PY

RASTER post_save
export_raster_to_cog()

VECTOR save/delete
bump_layer_version()
→ GeoJSON cache

WATERSUPPLY/SIGNALS.PY + SAVE()

Population (cascade)

ExtractionWater

ConsumptionCapita

ImportedWater

TotalWaterDemand

TotalWaterProduction

SupplySecurity

Service_Time

OPEX
extraction · imported
treatment · pipes

WaterTreatment · MeteredResidential · PipeNetwork → OPEX

NonRevenueWater
→ same-year ILI

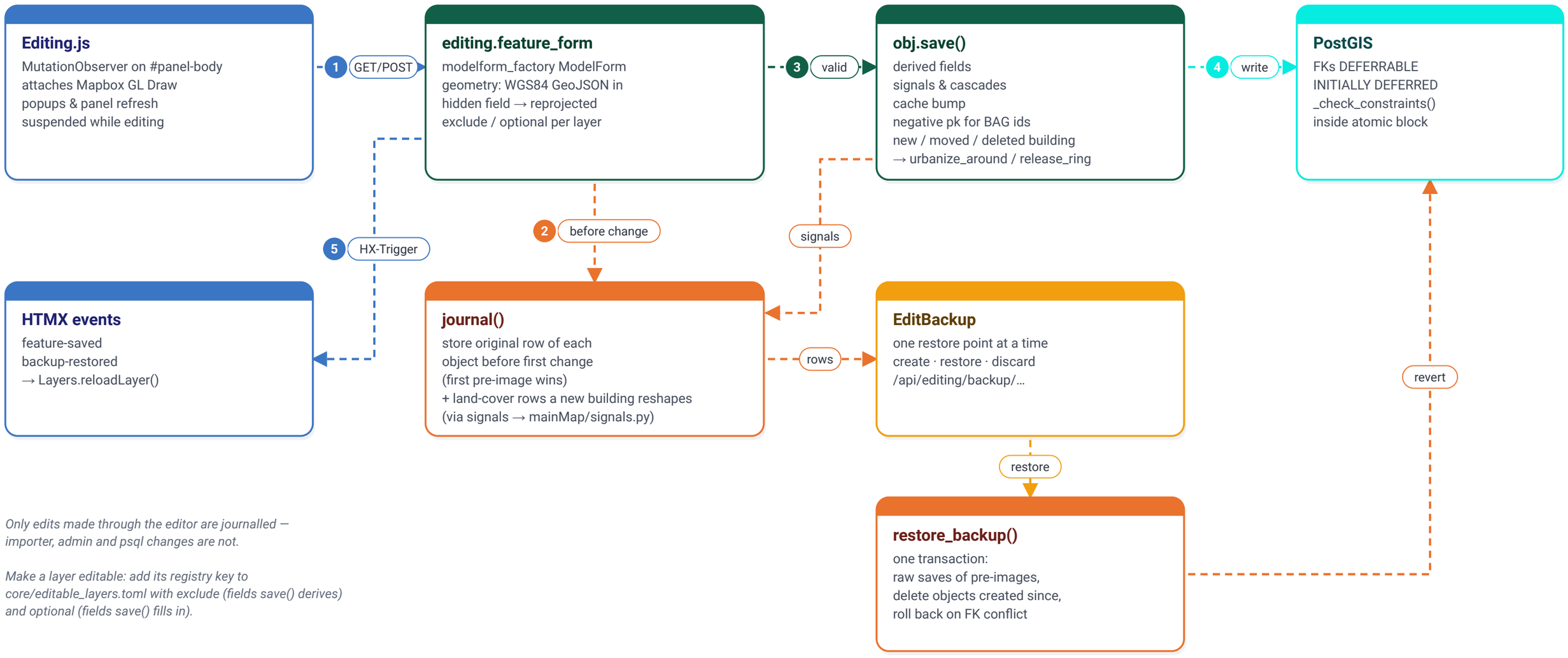
Coverage
PipeNetwork +
UsersLocation

BULK WRITES · IMPORTER/BATCHING.PY::DEFERRED_CASCADES()

= administrative.deferred_population_cascade() + physicalEnv.deferred_urban_area()
+ watersupply.deferred_water_refresh()
Inside the block, receivers only record the affected parent; on exit each cascade runs once per parent
(errors during that flush are logged, never raised over the original exception).
Always call schedule_urban_area_recompute() rather than the recompute itself, so an enclosing deferral is respected.

Map Editing & What-If Backup

Staff edit EDITABLE_LAYERS on the map; every change goes through obj.save() and is journalled for one-click restore

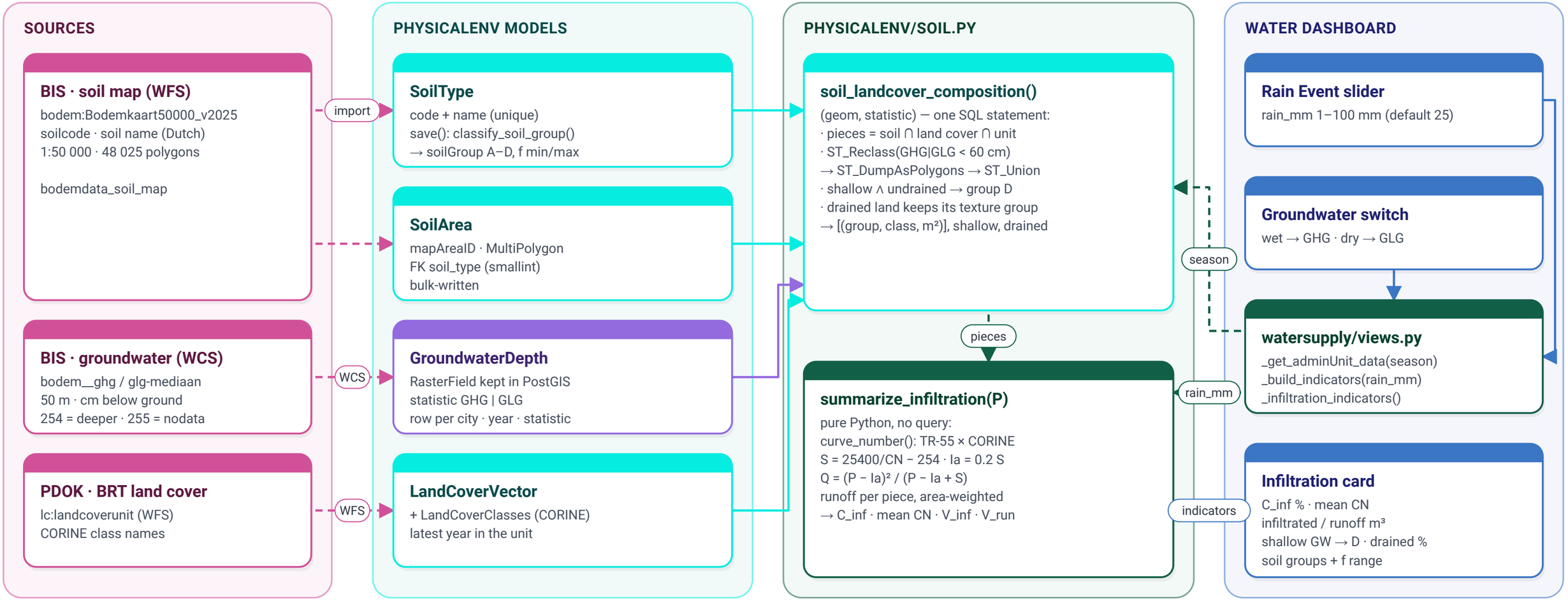


Only edits made through the editor are journalled — importer, admin and psql changes are not.

Make a layer editable: add its registry key to core/editable_layers.toml with exclude (fields save() derives) and optional (fields save() fills in).

Soil Hydrology & Infiltration — SCS Curve Number

physicalEnv/soil.py: soil map × land cover × groundwater → hydrologic soil group → curve number → infiltrated share of a rain event

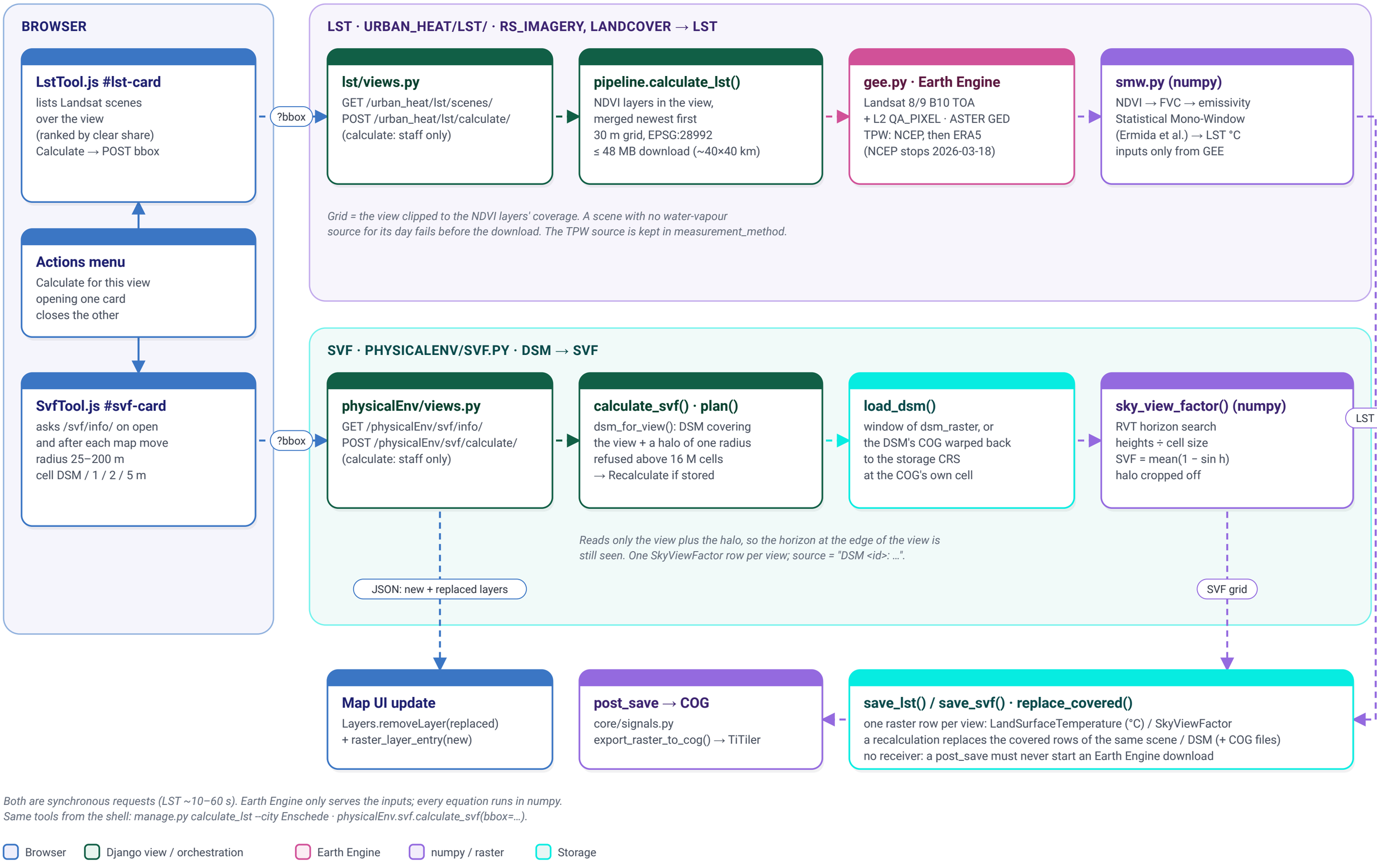


Soil group from the Dutch texture words in the soil name (zand A · zavel/leem B · lichte klei C · klei/veen D); the most restrictive layer wins.
NEH 630: a water table within 60 cm makes a soil group D; drained land covers (urban fabric, industrial, roads, ports, airports) keep their texture letter.
BIS has no GHG/GLG under most built-up land (85 % of urban-fabric cells in Enschede) or water: nodata counts as not shallow; water/wetland CN = 100/98.
The rain depth enters only the pure SCS functions; the spatial query depends on the season alone (cacheable, currently re-run per request).

External source · Vector model / SQL · Raster · Python · Dashboard

On-Demand Calculations — LST & Sky View Factor

Actions menu → Calculate for this view: the current map view is computed in numpy and stored as one raster row



Both are synchronous requests (LST ~10–60 s). Earth Engine only serves the inputs; every equation runs in numpy.
Same tools from the shell: manage.py calculate_lst --city Enschede · physicalEnv.svf.calculate_svf(bbox=…).

Browser · Django view / orchestration · Earth Engine · numpy / raster · Storage